

Data report on Airline Ontime 2009 Performance

ST2195 Coursework

Author: Lau Kian Heng Tommy

Introduction

Study on a dataset of flights and airline data from Harvard Dataverse, extracted from 2009 ASA Statistical Computing and Graphics Data Expo 2009: Airline on time data. For this analysis, I will be using the commercial flight data for year 2001 and 2002. The objective of this report is to answer the following questions:

1. When is the best time of day, day of the week, and time of year to fly to minimise delays?
2. Do older planes suffer more delays?
3. How does the number of people flying between different locations change over time?
4. Can you detect cascading failures as delays in one airport create delays in others?
5. Use the available variables to construct a model that predicts delays.

Data Exploration

To start, we are provided with 4 csv files along with the datasets, airports.csv , carriers.csv , plane-data.csv , variable-descriptions.csv . We can summarise all the variables into a table as followed:

Attribute	Description	Type	Attribute	Description	Type
Year	1987-2008	Int	DepDelay	departure delay, in minutes	int
Month	1-12	int	Origin	origin IATA airport code	string
DayofMonth	1-31	Int	Dest	destination IATA airport code	string
DayOfWeek	1-7	Int	Distance	in miles	int
DepTime	actual departure time (local, hhmm)	Int	TaxiIn	taxi in time, in minutes	int
CRSDepTime	scheduled departure time (local, hhmm)	Int	TaxiOut	taxi out time in minutes	int
ArrTime	actual arrival time (local, hhmm)	Int	Cancelled	was the flight cancelled?	int
CRSArrTime	scheduled arrival time (local, hhmm)	int	CancellationCode	reason for cancellation (A = carrier, B = weather, C = NAS, D = security)	int
UniqueCarrier	unique carrier code	string	Diverted	1 = yes, 0 = no	int
FlightNum	flight number	int	CarrierDelay	in minutes	int
TailNum	plane tail number	string	WeatherDelay	in minutes	int
ActualElapsedTime	in minutes	int	NASDelay	in minutes	int
CRSElapsedTime	in minutes	int	SecurityDelay	in minutes	int
AirTime	in minutes	int	LateAircraftDelay	in minutes	int
ArrDelay	arrival delay, in minutes	int			

Table 1: Dataset variables (variable-descriptions.csv)

Attribute	Description	Type	Attribute	Description	Type
Iata	Airport code	string	Country	Airport Country	string
Airport	Airport name	string	Lat	Airport Latitude	Int
City	Airport city	string	Long	Airport Longitude	int

Table 2: Airport data variables (airports.csv)

Attribute	Description	Type
Code	Carrier code	string
Description	Carrier name	string

Table 3: Carrier data variables (carriers.csv)

Attribute	Description	Type	Attribute	Description	Type
TailNum	Plane tail number	string	Status	Validity	int
Type	Usage type	string	Aircraft_type	Aircraft type	string
Manufacturer	Manufacturer	string	Engine_type	Engine type	string
Issue_date	Date of issue	string	Year	Manufactured year	int
Model	Aircraft model	string			

Table 4: Plane data variables (plane-data.csv)

Data wrangling process

Syntax used for wrangling the dataset is as followed **for Python**:

```
import sqlite3
import pandas as pd
import numpy as np
import os
try:
    os.remove('../Python files/py_airline.db')
except OSError:
    pass
conn = sqlite3.connect('../Python files/py_+airline.db')

: c = conn.cursor()

: c.execute('''
CREATE TABLE arrived (
    Year int,
    Month int,
    DayofMonth int,
    DayOfWeek int,
    DepTime int,
    CRSDepTime int,
    ArrTime int,
    CRSArrTime int,
    UniqueCarrier varchar(5),
    FlightNum int,
    TailNum varchar(8),
    ActualElapsedTime int,
    CRSElapsedTime int,
    AirTime int,
    ArrDelay int,
    DepDelay int,
    Origin varchar(3),
    Dest varchar(3),
    Distance int,
    TaxiIn int,
    TaxiOut int,
    Cancelled int,
    CancellationCode varchar(1),
    Diverted varchar(1),
    CarrierDelay int,
    WeatherDelay int,
    NASDelay int,
    SecurityDelay int,
    LateAircraftDelay int
)
''')
conn.commit()

for year in range(2001, 2002):
    print('Processing: ', year)
    arrived1 = pd.read_csv("../Python files/"+str(year)+".csv", encoding='latin-1')
    arrived1.to_sql('arrived1', con = conn, if_exists = 'append', index = False)
for year in range(2002, 2003):
    print('Processing: ', year)
    arrived2 = pd.read_csv("../Python files/"+str(year)+".csv", encoding='latin-1')
    arrived2.to_sql('arrived2', con = conn, if_exists = 'append', index = False)
conn.commit()

Processing: 2001
Processing: 2002
```

Import sqlite3 > define sqlite connection as conn > create a table database with c = conn.cursor() > Create tables for dataset and call it arrived > commit and save table variables into database > Use for loop and create table for 2001(defined as arrived1) and 2002 (defined as arrived2) data. Dataset will be retrieved using these syntaxes for every question.

Syntax used for wrangling the dataset in R is as followed:

```

'''{r}
# Creating the database
if (file.exists("r_airline5.db"))
  file.remove("r_airline5.db")

conn <- dbConnect(RSQLite::SQLite(), "r_airline5.db")
airports <- read.csv("airports.csv", header = TRUE)
carriers <- read.csv("carriers.csv", header = TRUE)
planes <- read.csv("plane-data.csv", header = TRUE)
dbwriteTable(conn, "airports", airports)
dbwriteTable(conn, "carriers", carriers)
dbwriteTable(conn, "planes", planes)
'''

```

```

'''{r}
##For loop to write a table for each year
for(i in c(2001)) {
  filename <- paste0(i, ".csv.bz2")
  print(paste("Processing:", i))
  arrived1 <- read.csv(filename, header = TRUE, encoding = "latin1", stringsAsFactor=FALSE)
  if(i == 2001) {
    dbwriteTable(conn, "arrived1", arrived1, append = TRUE)
  }
}
for(i in c(2002)) {
  filename <- paste0(i, ".csv.bz2")
  print(paste("Processing:", i))
  arrived2 <- read.csv(filename, header = TRUE, encoding = "latin1", stringsAsFactor=FALSE)
  if(i == 2002) {
    dbwriteTable(conn, "arrived2", arrived2, append = TRUE)
  }
}
arrived <- rbind(arrived1,arrived2)
'''

```

Create a database for tables by defining conn > create tables for airport carrier & plane variables > Use for loop and create table for 2001 and 2002 and rbind() to concatenate all data into one data frame (arrived).

Exploring the dataset

1. When is the best time of day, day of the week, and time of year to fly to minimise delays?

To being addressing the question, we have to define what is delay. The variables that we can include as delay is Arrival Delay , Departure Delay, Carrier Delay, Weather Delay, NAS Delay, Security Delay, Late Aircraft Delay. However in the study, it is filtered down to Arrival and Departure delay due to missing data in the other variables. Arrival and departure delays are duplicate, but for the purpose of the study only Arrival delay is considered because a passenger can depart late but still arrived on time.

To start with best time of day, we have the variable CRSDepTime or scheduled Departure time. I broke it down into 8 groups (0000–0259, 0300–0559, 0600–0859, ... etc) separated by 3-hour intervals. Referring to the code below from my python script (Qn1_in_Python), first I generate a table called “arrived” with all the data from 2001 and 2002. I called the DBI function ‘dr1a’ to COUNT all observations with delay between 0000 to 0259 hrs from the “arrived” table, and sum it all into one integer with ‘dr1a_a’ to generate a numerator. And subsequently ‘dr1b_a’, the total observations regardless delayed or not as the denominator. With these I called the function HR0002 to generate a mean value for delay.

```

In [33]: #dr1a to fetch all observations with delay between 0000 to 0259
dr1a = c.execute('''
SELECT COUNT(ArrDelay)
FROM arrived
WHERE CRSDepTime BETWEEN 0 AND 259 AND ArrDelay >0
GROUP BY CRSDepTime''').fetchall()

dr1a_a = sum(list(map(sum, list(dr1a)))) #Convert tuple output into integer #Numerator

#remove blank values and sums all possible observations
dr1b = c.execute('''
SELECT COUNT(ArrDelay)
FROM arrived
WHERE CRSDepTime BETWEEN 0 AND 259 AND ArrDelay IS NOT NULL
GROUP BY CRSDepTime''').fetchall()
dr1b_a = sum(list(map(sum, list(dr1b)))) #Convert tuple output into integer #Denominator

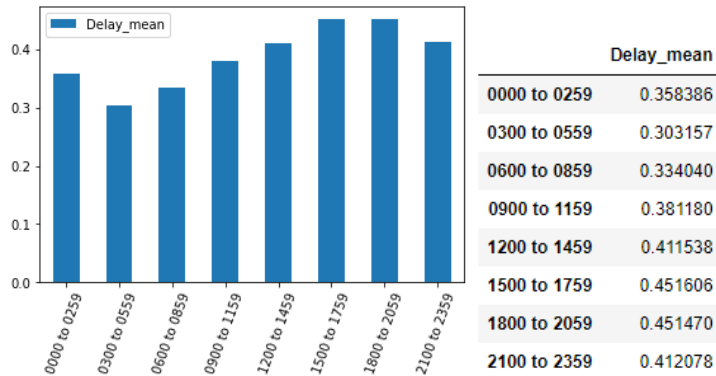
HR0002 = (dr1a_a/dr1b_a) # numerator / denominator
#HR0002 = mean delay rate between 0000hrs to 0259hrs

```

I called similar functions for the other 7 groups and visualised all these into a table with plot.bar function. With the data plotted we can observe that the **best time to fly would be between 0300 to 0559 hours**.

```
daily = {'Delay_mean':[HR0002,HR0305,HR0608,HR0911,HR1214,HR1517,HR1820,HR2123]}
daily_df = pd.DataFrame(daily, index =['0000 to 0259','0300 to 0559','0600 to 0859',
                                       '0900 to 1159','1200 to 1459','1500 to 1759',
                                       '1800 to 2059','2100 to 2359'])
daily_df
```

```
ax = daily_df.plot.bar(rot=70)
```



For best time of week to fly, I filter using the syntax WHERE DayOfWeek = 1 and similarly generate a numerator and denominator for the mean.

```
dr_Mo_a = c.execute('''
SELECT COUNT(ArrDelay)
FROM arrived
WHERE DayOfWeek = 1 AND ArrDelay > 0
GROUP BY DayOfWeek''').fetchall()
dr_Mon_a = sum(list(map(sum, list(dr_Mo_a))))

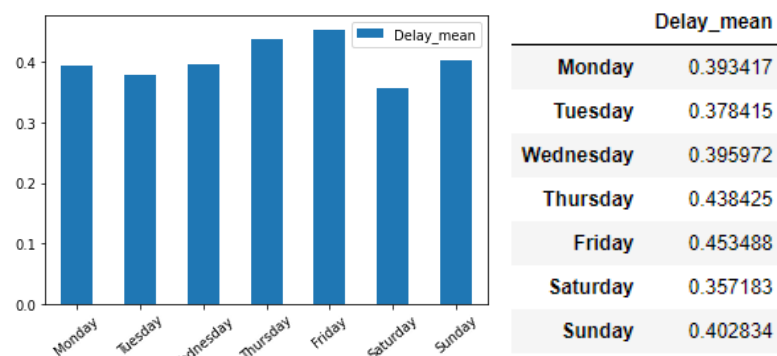
dr_Mo_b = c.execute('''
SELECT COUNT(ArrDelay)
FROM arrived
WHERE DayOfWeek = 1 AND ArrDelay IS NOT NULL
GROUP BY DayOfWeek''').fetchall()
dr_Mon_b = sum(list(map(sum, list(dr_Mo_b))))

Mondays = (dr_Mon_a/dr_Mon_b)
```

With 7 days in a week, I was able to generate a box plot with the following syntax:

```
weekday = {'Delay_mean':[Mondays,Tuesdays,Wednesday,Thursday,Friday,Saturday,Sunday]}
weekday_df = pd.DataFrame(weekday, index =['Monday','Tuesday','Wednesday','Thursday',
                                           'Friday','Saturday','Sunday'])
weekday_df
```

```
gx = weekday_df.plot.bar(rot=70)
```



With the bar plot we are able to observe that the **best day of week to fly would be on Saturdays**.

With a similar syntax, I switch the filter criteria Months where it is broken down to 12 groups, for each month of the year.

```
dr_Ja_a = c.execute('''
SELECT COUNT(ArrDelay)
FROM arrived
WHERE Month = 1 AND ArrDelay >0
GROUP BY Month''').fetchall()
dr_Jan_a = sum(list(map(sum, list(dr_Ja_a))))

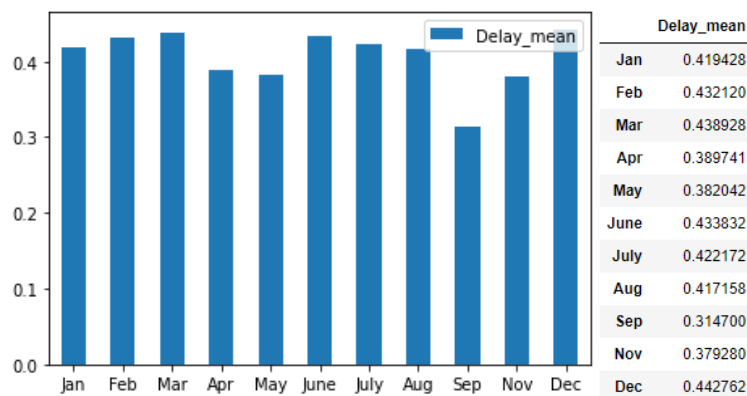
dr_Ja_b = c.execute('''
SELECT COUNT(ArrDelay)
FROM arrived
WHERE Month = 1 AND ArrDelay IS NOT NULL
GROUP BY Month''').fetchall()
dr_Jan_b = sum(list(map(sum, list(dr_Ja_b))))

Jan = (dr_Jan_a/dr_Jan_b)
Jan
```

Generating a table for each month of the year, I was able to plot the following graph

```
Month = {'Delay_mean':[Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Nov, Dec]}
Month_df = pd.DataFrame(Month, index = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'June',
                                         'July', 'Aug', 'Sep', 'Nov', 'Dec'])
Month_df

ax = Month_df.plot.bar(rot=0)
```



From this we can observe that the **best time of the year to fly would be in Septembers.**

2. Do older planes suffer more delays?

Using Python (Qn2_in_Python) with the following syntax, I create table using issue_date from planes.csv as Age, and I use JOIN function to match it with data in “arrived1” using TailNum, to create the Table df1 (refer to figure). I created df1 which represents 2001 data and df2 for 2002 data, with that I convert them from a list to dataframe and concatenated them into df3. I also cleaned the data with the “df3 = dt3.mask(dt3.astype(object).eq('None')).dropna()” function.

```
df1 = c.execute('''
SELECT issue_date AS Age, arrived1.ArrDelay as Delay
FROM planes JOIN arrived1 USING(TailNum)
WHERE ArrDelay >0
GROUP BY Age
ORDER BY Delay DESC
''').fetchall()

df2 = c.execute('''
SELECT issue_date AS Age, arrived2.ArrDelay as Delay
FROM planes JOIN arrived2 USING(TailNum)
WHERE ArrDelay >0
GROUP BY Age
ORDER BY Delay DESC
''').fetchall()

dt1 = pd.DataFrame(df1)
dt2 = pd.DataFrame(df2)
dt3 = pd.concat([dt1, dt2])
dt3.columns = ['Age', 'Delay']
df3 = dt3.mask(dt3.astype(object).eq('None')).dropna() #Clean data and drop NAs
```

	Age	Delay
0	5/13/1998	441.0
1	10/8/1998	216.0
2	4/12/1995	191.0
3	6/1/1994	175.0
4	8/3/1998	167.0
5	8/10/2001	108.0
6	6/2/2000	102.0
7	12/9/1994	102.0
8	6/29/2001	99.0
9	8/14/1997	96.0
10	2/21/1995	93.0

Figure 1. Table df1 ↑

With the following syntax, I converted the Age format from Figure 1 into age numbers which can be visualised from Figure 3 shown later.

```
def dob(born):
    born = datetime.strptime(born, "%m/%d/%Y").date()
    today = date.today()
    return today.year - born.year - ((today.month,
                                     today.day) < (born.month,
                                                     born.day))

df3['Age'] = df3['Age'].apply(dob)
```

Using the following syntax, using sort.values, I grouped the data by ascending Age and plotted the following boxplot graph:

```
df4 = df3.sort_values(by='Age', ascending=True)
ax = df4.groupby('Age').size().plot(kind='bar',
                                     title='Delay by age',
                                     xlabel="Age",
                                     ylabel="Delay")
plt.show()
```

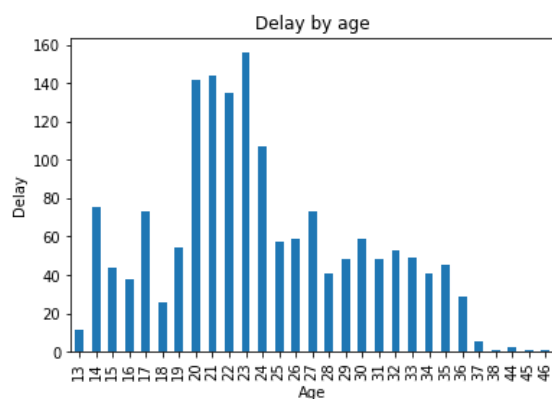


Figure 2. Boxplot of Delay by age\

Since it's not conclusive with just boxplot, we need to explore the data further. I wrote the following syntax, transposing and disabling column limit for easier visualisation.

```
pd.set_option('display.max_columns', 2000)
dt4 = np.transpose(df4)
dt4
```

	970	418	228	1322	859	67	502	73	388	215	24	1032	277	830	824	817	1122	801	304	309	795	1274	318	1146	787	1282
Age	13.0	13.0	13.0	13.0	13.0	13.0	13.0	13.0	13.0	13.0	13.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0	14.0
Delay	6.0	22.0	2.0	1.0	8.0	31.0	18.0	78.0	23.0	3.0	139.0	5.0	31.0	9.0	9.0	9.0	4.0	10.0	29.0	28.0	10.0	2.0	28.0	3.0	10.0	2.0

Figure 3.1. Boxplot of Delay by age

	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	24.0	
	92.0	2.0	3.0	2.0	30.0	17.0	30.0	22.0	9.0	9.0	22.0	1.0	194.0	13.0	23.0	15.0	36.0	28.0	39.0	1.0	31.0	4.0	1.0	7.0	32.0	4.0	13.0	13.0

Figure 3.2. Boxplot of Delay by age

	13	125	1031	77	94	139	84	250	72	69	65	382	118	781	254	123	52	1341	421	842	431	427	753	200	1049	1318	127	188	
	.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	22.0	
	.0	13.0	5.0	26.0	21.0	53.0	73.0	33.0	29.0	30.0	84.0	23.0	14.0	10.0	33.0	56.0	96.0	1.0	22.0	9.0	21.0	22.0	10.0	4.0	5.0	1.0	12.0	5.0	3

Figure 3.3. Boxplot of Delay by age

From figure 3.1 we can see the highest delay at 139.0 and there isn't an obvious trend from the surrounding data. At figure 3.2, where the delay is the highest at Age 24, we can see that the delayed plane is subjective, since the data around it is similar to Fig 3.1, with most under 10min delays and occasional 20 to 30min delays. Even in Fig 3.3, it seems like a temporary trend and is inconclusive. **In conclusion, there isn't sufficient evidence point the correlation of Age to Flight delays.**

3. How does the number of people flying between different locations change over time?

Using Rstudio (Qn3_in_R.rmd), I loaded the csv and created tables arrived1 for 2001 data and 2002 data, with the syntax from Fig. 4.1, I took the columns Origin and Destination and wrote it into dataframe (df1). Using dplyr I piped the function count() to generate a count of pairs of Origin airport to their Destination airport and insert count as a 3rd column "n". I ordered the column by Origin and descending, and renamed the columns to Flight_count illustrated in Figure 4.2.

```
##{r}
#Create a table for 2001
df1 <- data.frame(Origin = c(arrived1$Origin), Destination = c(arrived1$Dest))
df2 <- df1 %>%
  count(Origin, Destination, sort = TRUE)
df2 = df2[order(df2$Origin),]
df3 <- df2 %>%
  rename(Flight_count = n)
##
```

	Origin	Destination	Flight_count
1384	ABE	PIT	1454
1690	ABE	ORD	1157
2505	ABE	DTW	621
2525	ABE	ATL	607
2551	ABE	MCO	574
2644	ABE	MDT	485
2905	ABE	CLT	364
748	ABE	DFW	2567

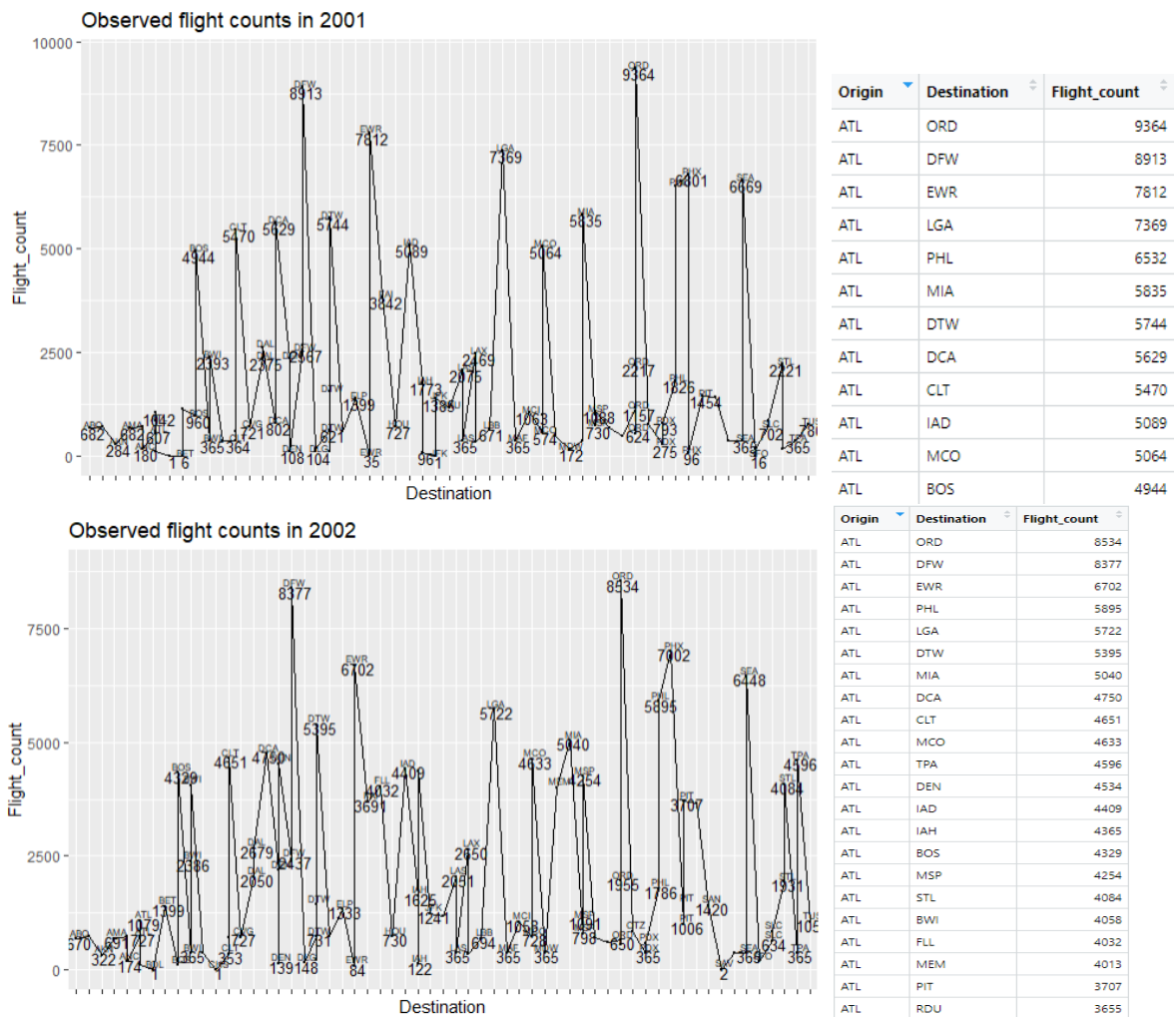
Figure 4.1. Syntax for pair counts

Figure 4.2. Table illustrating syntax

After using the same formula to generate a similar syntax for 2002 data, I proceeded to plot a line graph with geom_line() in ggplot2 package. Due to the huge data size, I'm using only the first 100 observations. Setting Destination as the dependent variable, and Flight_count as the independent, to plot the following graphs.

```
##{r}
#Flights for 2001
df4 <- df3 %>%
  head(100)
ggplot(df4, aes(x=Destination, y=Flight_count, group = 1)) +
  ggtitle("Observed flight counts in 2001") +
  geom_line() +
  geom_point(size=0.25) +
  geom_text(
    label=df4$Flight_count,
    nudge_x = 0.25, nudge_y = 25,
    size=3.5,
    vjust=1,
    check_overlap = T
  ) +
  geom_text(
    label=df4$Destination,
    nudge_x = 0.25, nudge_y = 200,
    size=2,
    vjust=1,
    check_overlap = T
  ) +
  theme(axis.text.x = element_blank())
print(as_tibble(df3))
```

From the graphs below, looking at just ATL, we can observe that the flights from Atlanta airport (ATL) have decreased from 2001 to 2002, and major flight destinations towards Dallas Texas (DFW) and Chicago (ORD) has decreased. Relatively to that, flights seem to have increase and traffic is diverted to Philadelphia (PHL) and Pittsburgh (PIT), with more destinations flying away from major cities. This could be due to the September 11 attacks that occurred in 2001, which diverted traffic away to destinations such as Minneapolis (MSP) which is further away from Washington city. More exploration can be done on other airports, but we can observe a trend from the dataset.



4. Can you detect cascading failures as delays in one airport create delays in others

To answer this question, I will be using RStudio (refer to Qn4_in_R). I start with this syntax and group my data by Tail number (TailNum) to write the dataframe (df1).

```
##{r}
df1 <- select(arrived, Year, Month, DayOfMonth, DayOfWeek, TailNum, DepTime, CRSDepTime, ArrTime,
CRSArrTime, FlightNum, ActualElapsedTime, CRSElapsedTime, ArrDelay, DepDelay, Origin, Dest) %>%
  filter(DepDelay>0, ArrDelay>0) %>%
  group_by(TailNum)
```

Because Tail number is in Latin, R doesn't know how to sort it. So I wrote the following syntax to convert Latin to alphanumeric values. Using `iconv()`, I get an table with the corrected Tail number data, and convert it into a dataframe (df2). After that I replace the Tail number in the old dataframe and replace and removed it to get dataframe (df3).

```
##{r}
#Convert latin characters in df1 to ASCII
dt <- iconv(df1$TailNum, "latin1", "ASCII", sub="")
df2 <- data.frame(TailNum = c(dt))
df1$TailNum <- df2$TailNum #Add the correction into df1
df3 <- select(df1, Tail_Num, Year, Month, DayOfMonth, DayOfWeek, DepTime, CRSDepTime, ArrTime,
CRSArrTime, Origin, Dest, ActualElapsedTime, CRSElapsedTime, FlightNum, ArrDelay, DepDelay)
df3$TailNum <- NULL #Replace the TailNum column with correction
##
```

After getting df3, I wrote the following syntax to get the Tail number with the highest frequency of delays, and filter half of it out at the n>1400 point since there's a lot of observations.

```

{r}
dt2 <- count(df3,Tail_Num) #Count number of delay occurrences
df4 <- df3 %>%
  inner_join(dt2, by = "Tail_Num") %>%
  filter(n>1400)
df4 = df4[order(-df4$n),]
print(as_tibble(df4))

```

For the purpose of this report, I will be observing only carrier N5CA1. With the following syntax, I filter to N5CA1 by tail number, and sorted the values to give the tibble Figure 5. We can observe that the flight from LAX to MIA was delayed 32 minutes on arrival, after arriving at 1800hrs, it departed at 1920 to DCA and was subsequently late on arrival. We can dig up similar data but to answer the question, **it is possible to detect cascading delays**, as the delays can be associated with short maintenance time, or lack of maintenance due to small airports with limited garages for maintaining airplanes. Given a better skillset, it is possible to analyse further.

```

{r}
Carrier_N5CA1 <- df4 %>% #Filter to N5CA1
  filter(Tail_Num=="N5CA1")
Carrier_N5CA1 = Carrier_N5CA1[order(Carrier_N5CA1$DayOfMonth),]
print(as_tibble(Carrier_N5CA1))

```

RStudio: Notebook Output

A tibble: 2,765 x 17

Tail_Num	Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	Origin	Dest	ActualElapsedTime	CRSElapsedTime	FlightNum	ArrDelay	DepDelay	
N5CA1	2001	1	1	1	1016	930	1800	1728	LAX	MIA	284	298	280	32	46	
N5CA1	2001	1	1	1	1920	1915	2200	2142	MIA	DCA	160	147	720	18	5	
N5CA1	2001	1	1	1	1915	1730	2215	2033	MIA	LGA	180	183	860	102	105	
N5CA1	2001	1	1	1	1246	1150	1600	1511	EWB	MIA	194	201	1475	49	56	
N5CA1	2001	1	1	1	1217	1158	1807	1752	SNA	ORD	230	234	1698	15	19	
N5CA1	2001	1	1	1	716	630	1040	941	LGA	MIA	204	191	1731	59	46	
N5CA1	2001	1	1	1	1250	1248	1406	1357	DFW	AUS	76	69	1961	9	2	
N5CA1	2001	1	1	1	649	648	1209	1153	SNA	DFW	200	185	1961	16	1	
N5CA1	2001	1	1	1	1719	1530	2103	1916	JAC	ORD	164	166	2352	107	109	
N5CA1	2001	1	1	1	2218	2030	155	33	ORD	MIA	157	183	2352	82	108	
N5CA1	2001	1	1	1	1130	830	1321	1044	MIA	ORD	171	194	2353	157	180	
N5CA1	2001	1	1	1	1432	1145	1613	1410	ORD	JAC	161	205	2353	123	167	
N5CA1	2001	2	1	4	1315	1247	1611	1603	DFW	ATL	116	136	536	8	28	
N5CA1	2001	2	1	4	1727	1610	2109	2000	DFW	MIA	162	170	786	69	77	
N5CA1	2001	2	1	4	1703	1653	1821	1817	ATL	DFW	138	144	1541	4	10	
N5CA1	2001	2	1	4	1940	1919	2105	2045	DFW	TUS	145	146	1541	20	21	

1-16 of 2,765 rows | 1-14 of 17 columns

Previous 1 2 3 4 5 6 ... 63 Next

Figure 5. Tibble on N5CA1

5. Use the available variables to construct a model that predicts delays.

I will be using Python (refer to Qn5_in_Python) to answer this question. For the purpose of this question, I extract out variables that can classify into two categories, numerical and categorical, using SELECT from DBI and concatenate both df1 for 2001 and df2 into one dataframe 'dt3'.

```

df1 = c.execute('
SELECT Year, Month, DayOfWeek, DepDelay, Cancelled, Diverted,
ActualElapsedTime, CRSElapsedTime, AirTime, Distance, TaxiIn, TaxiOut
FROM arrived1
WHERE Year IS NOT 2003
').fetchall()

df2 = c.execute('
SELECT Year, Month, DayOfWeek, DepDelay, Cancelled, Diverted,
ActualElapsedTime, CRSElapsedTime, AirTime, Distance, TaxiIn, TaxiOut
FROM arrived2
WHERE Year IS NOT 2003
').fetchall()

dt1 = pd.DataFrame(df1)
dt2 = pd.DataFrame(df2)
dt3 = pd.concat([dt1, dt2])
dt3.columns = ['Year', 'Month', 'Weekday', 'DepDelay', 'Cancelled', 'Divered',
'ActualElapsedTime', 'CRSElapsedTime', 'AirTime', 'Distance', 'TaxiIn', 'TaxiOut']

```

Cleaned the data, removing infinity values, NAN, and NA values, and generate a sample size of 100 000, giving me 'df3'.

```
#Cleaning dataset
dt3.replace([np.inf, -np.inf], np.nan, inplace=True)
dt3.fillna(0, inplace=True)
#Generating sample size
df3 = dt3.sample(100000)
```

Added a new feature using where clause when delay is more than 0 and present, the value is 1 (TRUE), and if not the value is 0 (FALSE). Df3.Nunique() tells us that only two values 0 or 1 is present in 'Delayed'.

```
: #Add a new feature Delayed
df3['Delayed'] = np.where(df3['DepDelay']>0, 1, 0)
df3['Diverted'] = df3['Diverted'].astype('int')
from collections import Counter
df3.nunique()

: Year                2
  Month              12
  Weekday            7
  DepDelay          203
  Cancelled          2
  Diverted           2
  ActualElapsedTime  381
  CRSElapsedTime    374
  AirTime           361
  Distance          1013
  TaxiIn            57
  TaxiOut           91
  Delayed           2
dtype: int64
```

With the following syntax, I exclude DepDelay since it's just a duplicate for the feature 'Delayed', and set the feature 'Delayed' as the independent variable Y.

```
#Pre-processing stage, defining X and Y for train set
features = ['Year', 'Month', 'Weekday', 'Cancelled', 'Diverted',
'ActualElapsedTime', 'CRSElapsedTime', 'AirTime', 'Distance', 'TaxiIn', 'TaxiOut']
X = df3[features].copy()
y = np.array(df3['Delayed'])
```

I split all variables into numerical and categorical features. For the numerical features, I apply SimpleImputer to replace missing values, StandardScaler to scale all values to mean 0 and standard deviation 1 for ease of computing, and MinMaxScaler to scale all the features to range between 0 and 1.

```
numerical_features = ['ActualElapsedTime', 'CRSElapsedTime', 'AirTime', 'Distance', 'TaxiIn', 'TaxiOut']

# Applying SimpleImputer, StandardScaler and MinMaxScaler into a pipeline
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler()),
    ('minmax', MinMaxScaler())])

categorical_features = ['Year', 'Month', 'Weekday', 'Cancelled', 'Diverted']

# Applying SimpleImputer and then OneHotEncoder into another pipeline
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer()),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))])

data_transformer = ColumnTransformer(
    transformers=[
        ('numerical', numerical_transformer, numerical_features),
        ('categorical', categorical_transformer, categorical_features)])
```

With the following syntax, I split the train and test sets into 50% each and set the parameters for numerical and categorical.

```
#Creating test and train sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=1)
```

```
#Setting parameters
param_grid = {
    'data_transformer__numerical__imputer__strategy': ['mean', 'median'],
    'data_transformer__categorical__imputer__strategy': ['constant', 'most_frequent']
}
```

Using sklearn module, I train the data and perform the tasks Logistic regression, Gradient boosting, Classification tree, Random forest, and Support vector machines test.

```
#Logistic regression
pipe_lr = Pipeline(steps=[('data_transformer', data_transformer),
                          ('pipe_lr', LogisticRegression(max_iter=10000, penalty='none'))]) #penalty='l1'

grid_lr = GridSearchCV(pipe_lr, param_grid=param_grid)
grid_lr.fit(X_train, y_train);

#Gradient Boosting
pipe_gdb = Pipeline(steps=[('data_transformer', data_transformer),
                          ('pipe_gdb', GradientBoostingClassifier(random_state=2))])

grid_gdb = GridSearchCV(pipe_gdb, param_grid=param_grid)
grid_gdb.fit(X_train, y_train);

# Classification tree
pipe_tree = Pipeline(steps=[('data_transformer', data_transformer),
                          ('pipe_tree', DecisionTreeClassifier(random_state=0))])
grid_tree = GridSearchCV(pipe_tree, param_grid=param_grid)
grid_tree.fit(X_train, y_train);

# Random forest
pipe_rf = Pipeline(steps=[('data_transformer', data_transformer),
                          ('pipe_rf', RandomForestClassifier(random_state=0))])
grid_rf = GridSearchCV(pipe_rf, param_grid=param_grid)
grid_rf.fit(X_train, y_train);

# Support vector machines
pipe_svm = Pipeline(steps=[('data_transformer', data_transformer),
                          ('pipe_svm', LinearSVC(random_state=0, max_iter=10000, tol=0.01))])
grid_svm = GridSearchCV(pipe_svm, param_grid=param_grid)
grid_svm.fit(X_train, y_train);
```

The values predicted by the machine can be visualised with the following syntax, which plots a model for each task visualised in figure 5.

```
ax, fig = plt.subplots(figsize=(12,12))
ax = plt.gca()

plot_roc_curve(grid_lr, X_test, y_test, ax=ax, name='Logistic Regression')
plot_roc_curve(grid_gdb, X_test, y_test, ax=ax, name='Gradient Boosting')
plot_roc_curve(grid_tree, X_test, y_test, ax=ax, name='Classification trees')
plot_roc_curve(grid_rf, X_test, y_test, ax=ax, name='Random forests')
plot_roc_curve(grid_svm, X_test, y_test, ax=ax, name='Support vector machines')
plt.plot([0, 1], [0, 1], color='black', lw=1, linestyle='--')
plt.show()
```

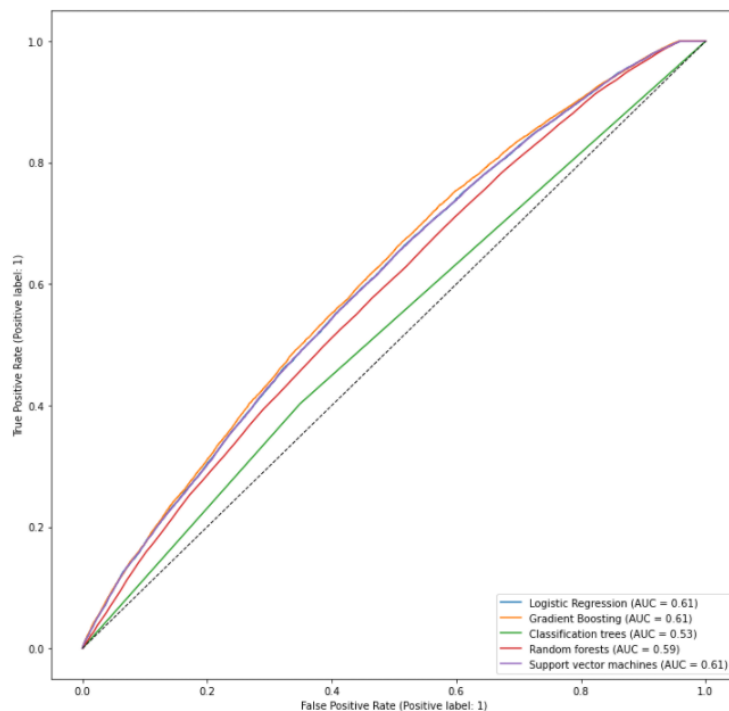


Figure 5. ROC plot on prediction models

Conclusion

A higher Area Under Curve (AUC) value will give more accurate predictions, which we can use for hyperparameter tuning. To answer the question, we can use Logistic regression, Gradient boosting, or Support vector machines (SVM) models to tune the hyperparameters to accurately predict flight delays.